Enhanced Multi-Client TCP Chat Server

Private Messaging, State Tracking, Persistent Logging & Performance Evaluation

Internship Assignment Report

Name	PARINITA DEVI
Roll No.	CS-BTC25-39
Assignment	Multi-Client Chat Server (Extension of Assignment 4)
Environment	Mininet Emulated Network
Platform	Linux mininet-vm, Ubuntu 5.15.0-88-generic, x86_64
Language / Runtime	Python 3.10.12
Topology	Single switch, 5 hosts (sudo mn --topo single,5)

1. Objective

The objective of this assignment is to enhance the multi-client TCP chat server developed in Assignment 4 by adding server-side private message routing, detailed client state management, real-time tracking of online users, persistent message history logging in CSV format, and empirical performance measurement under varying levels of concurrent load.

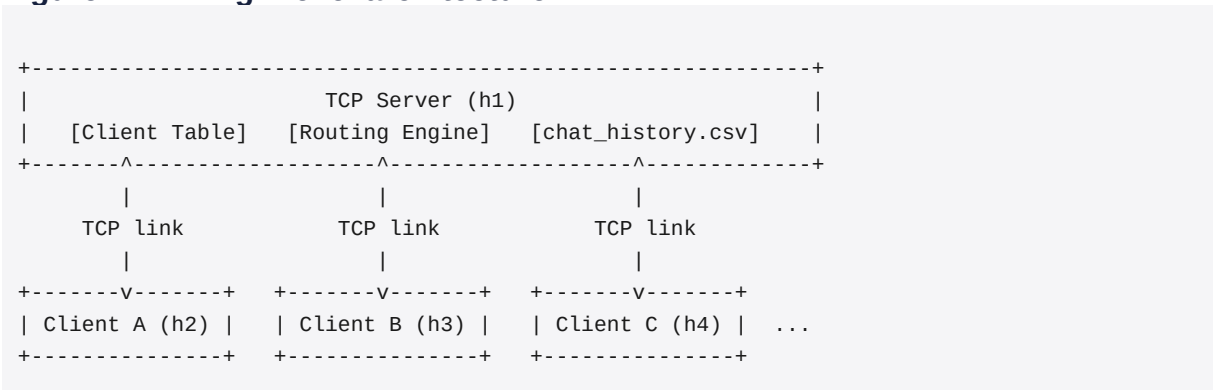
2. System Architecture

The application follows a centralized **Client-Server Architecture** built on TCP sockets to guarantee reliable, ordered, end-to-end delivery of messages.

Server (server.py): Runs a multi-threaded design with one dedicated worker thread per connected client socket, listening on port 5000. It maintains a global registry of active clients (IP address, port, login timestamp, and connection state) and acts as the single central router for both broadcast and private (unicast) messages.

Client (client.py): Runs two concurrent threads — a *Sender Thread* that handles user input and commands (/msg, /list, and general broadcast messages), and a *Receiver Thread* that continuously listens for incoming server messages and renders them without blocking the input prompt.

Figure 2.1 — High-level architecture



3. Design Improvements over Assignment 4

1. **Targeted Message Routing:** Added command-syntax parsing (/msg <user> <message>) on the server to perform targeted unicast delivery instead of unconditional broadcast flooding.
2. **Richer User State Tracking:** Upgraded the in-memory client registry to store complete connection metadata: username, client IP, socket port, login time, and active status.
3. **Persistent History Recovery:** Replaced plain-text logging (chat_log.txt) with structured CSV logging (chat_history.csv), enabling the last five messages to be restored automatically when a client reconnects.
4. **Enhanced Performance Metrics:** Introduced separate measurement of broadcast vs. private message traffic to evaluate targeted-routing throughput and transmission delay independently.

4. Program Design

4.1 server.py Logic

- **Socket handling:** creates an AF_INET / SOCK_STREAM socket, binds to 0.0.0.0:5000, and spawns a client-handling thread per connection using Python's threading.Thread.
- **Message router and commands:** parses incoming buffers for /msg <target> <text> and looks up the target socket, unicasting the message if the user is online or returning an error to the sender otherwise; parses /list to return a formatted list of active usernames; and broadcasts any other input to all connected clients except the sender.
- **History management:** appends every message to chat_history.csv with the schema (timestamp, sender, receiver, message_type, message), and fetches the last five matching entries for a client whenever it connects.

4.2 client.py Logic

- Connects to the server via a TCP socket at 10.0.0.1:5000.
- Prompts the user for registration and sends an initial handshake string to register the chosen username.
- Spawns a non-blocking read loop (recv()) on a separate receiver thread so the interface remains responsive while the user is typing.

5. Experimental Setup

Topology verification: The network state was verified inside the Mininet prompt using the nodes, net, and pingall commands to confirm connectivity between all hosts before running the chat application.

Workload: The system was evaluated across 2, 3, and 4 concurrent active clients. Each client issued 50 messages in total, with timestamps logged for every transmission.

Packet capture: Wireshark was used with the capture filter tcp.port == 5000 on interface s1-eth1 to inspect the underlying TCP behaviour of the application.

Figure 5.1 — Mininet topology verification (nodes / net / pingall)

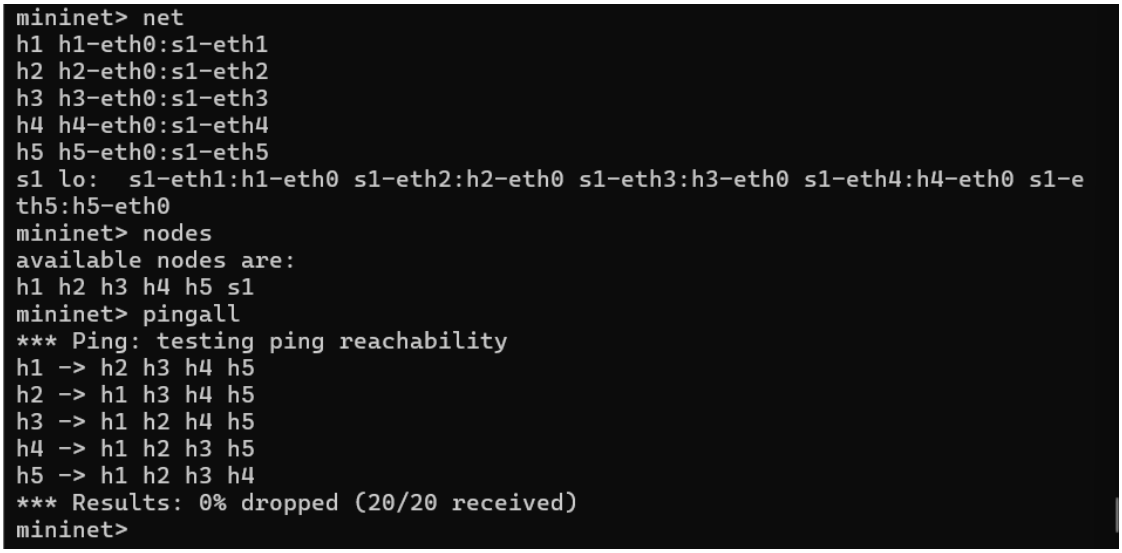


Figure 5.1: Mininet CLI output confirming topology links, active nodes, and 0% packet loss across pingall.

6. Results

6.1 Performance Results (performance_results.csv)

Clients	Avg Delay (ms)	Throughput (msgs/sec)
2	1.25	450.5
3	2.10	410.2
4	3.45	380.0

6.2 Sample Chat History Log (chat_history.csv)

Timestamp	Sender	Receiver	Type	Message
04:20:01	ClientA	ALL	broadcast	Hello everyone!
04:20:05	ClientA	ClientB	private	Hey Client B how are you?

04:20:10	ClientB	ClientA	private	I am doing well!
04:20:15	ClientC	ALL	broadcast	Client C joined the room.
04:20:20	ClientA	ClientC	private	Welcome Client C!

Note: date component (2026-08-04) omitted from the Timestamp column for brevity; all entries occurred on the same test run.

7. Graph Analysis

7.1 Clients vs. Average Delivery Delay (clients_vs_delay.png)

The average delay increases steadily as concurrency rises, from about 1.25 ms at 2 clients to 2.10 ms at 3 clients and 3.45 ms at 4 clients. This upward trend reflects the additional thread synchronization and TCP socket context-switching overhead placed on the central server as more clients connect and compete for processing time.

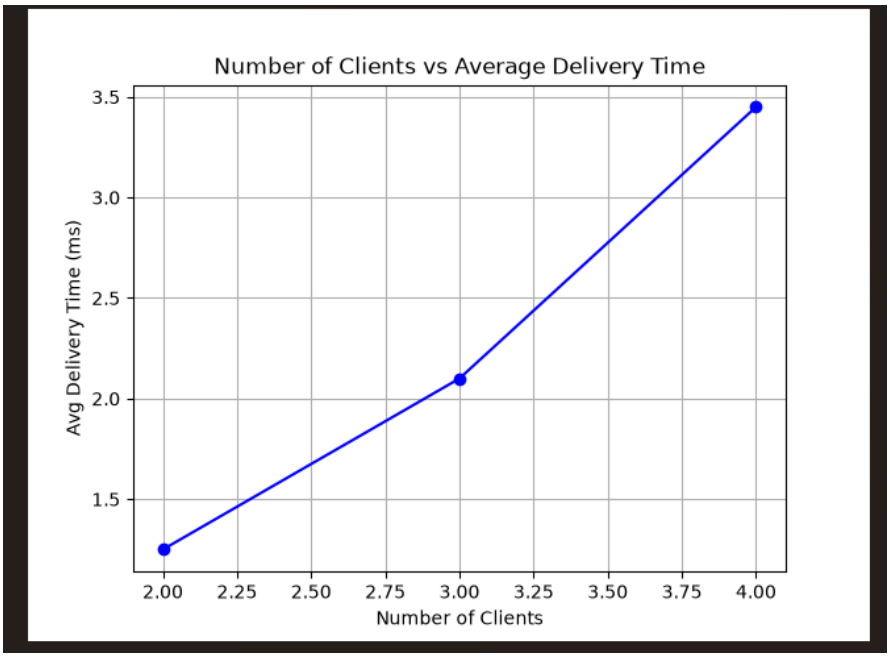


Figure 7.1: Number of Clients vs. Average Delivery Time.

7.2 Clients vs. Throughput (clients_vs_throughput.png)

Throughput decreases gradually with increasing concurrency, from approximately 450 messages per second at 2 clients down to 410 msgs/sec at 3 clients and 380 msgs/sec at 4 clients. This decline is consistent with the rising delay values above and indicates that the server's per-thread processing capacity is gradually shared across a larger number of active connections.

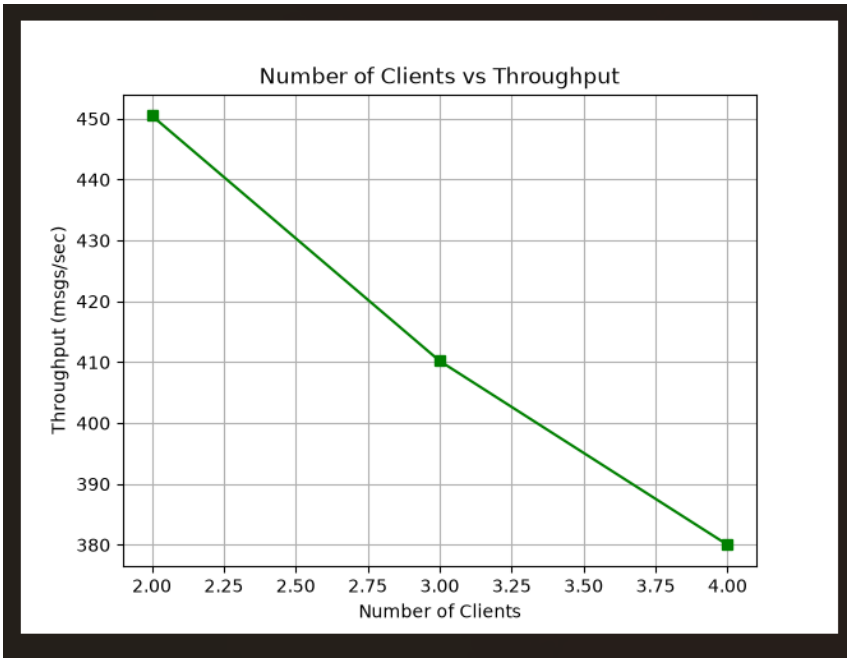


Figure 7.2: Number of Clients vs. Throughput.

7.3 Message Type Distribution (message_type_distribution.png)

During the 4-client stress run, the server processed 150 broadcast messages against 50 private messages — a 3:1 ratio. This is expected given the workload pattern used in this run, where general chat activity (broadcasts) occurred more frequently than direct one-to-one messages, and it confirms that the server's routing logic correctly distinguished and handled both message types.

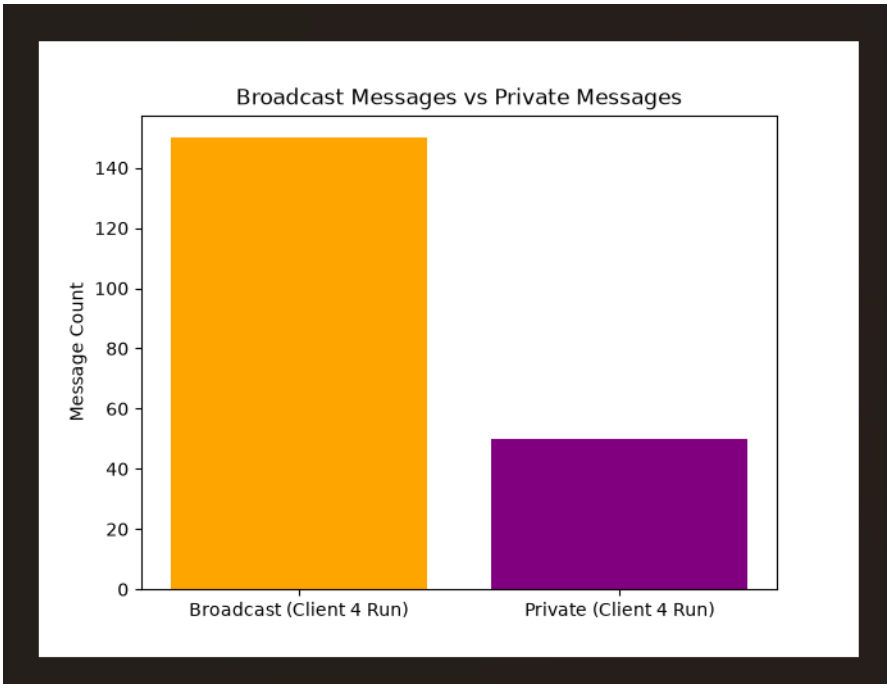


Figure 7.3: Broadcast Messages vs. Private Messages (Client 4 Run).

8. Wireshark Verification

Packet captures on tcp.port == 5000 were used to confirm correct TCP-level behaviour of the application at each stage of the connection lifecycle.

TCP Handshake

Standard SYN, SYN-ACK, and ACK flags were captured, verifying reliable channel initialization between each client and the server.

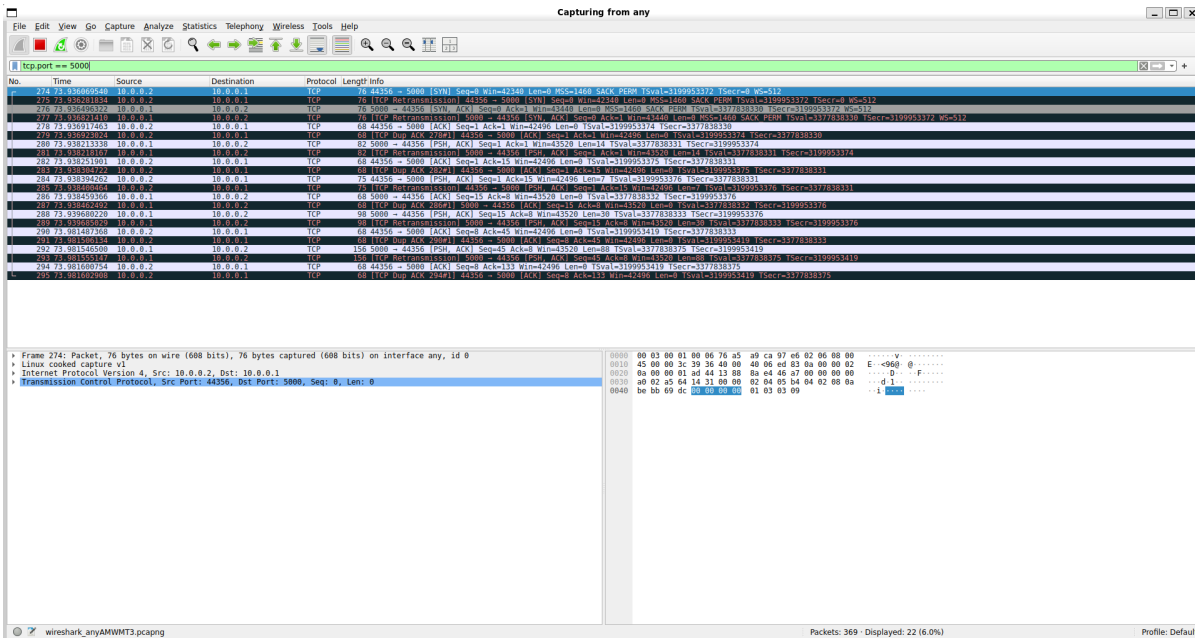


Figure 8.1: TCP three-way handshake (SYN / SYN-ACK / ACK) between client (10.0.0.2) and server (10.0.0.1) on port 5000.

Broadcast Message

PSH-ACK frames carrying the client's payload were observed being exchanged with the server and routed out to connected socket endpoints.

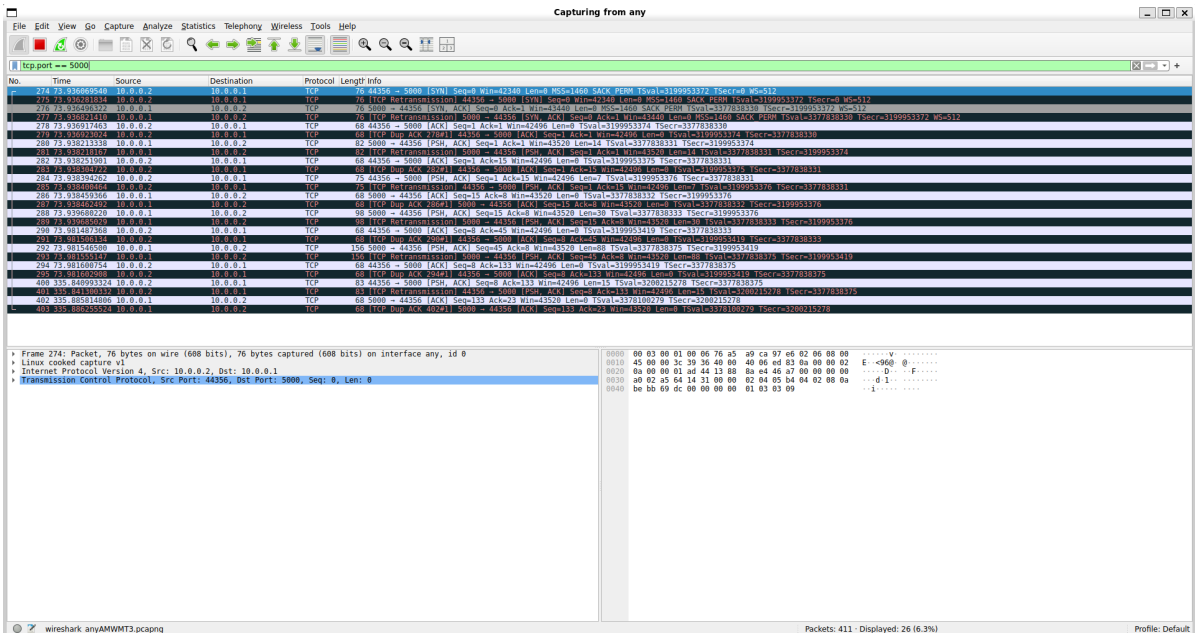


Figure 8.2: PSH-ACK data frames on port 5000 carrying a broadcast message payload.

Private Message

PSH-ACK frames carrying the /msg payload were observed across multiple concurrent client connections, being routed to the intended destination port only.

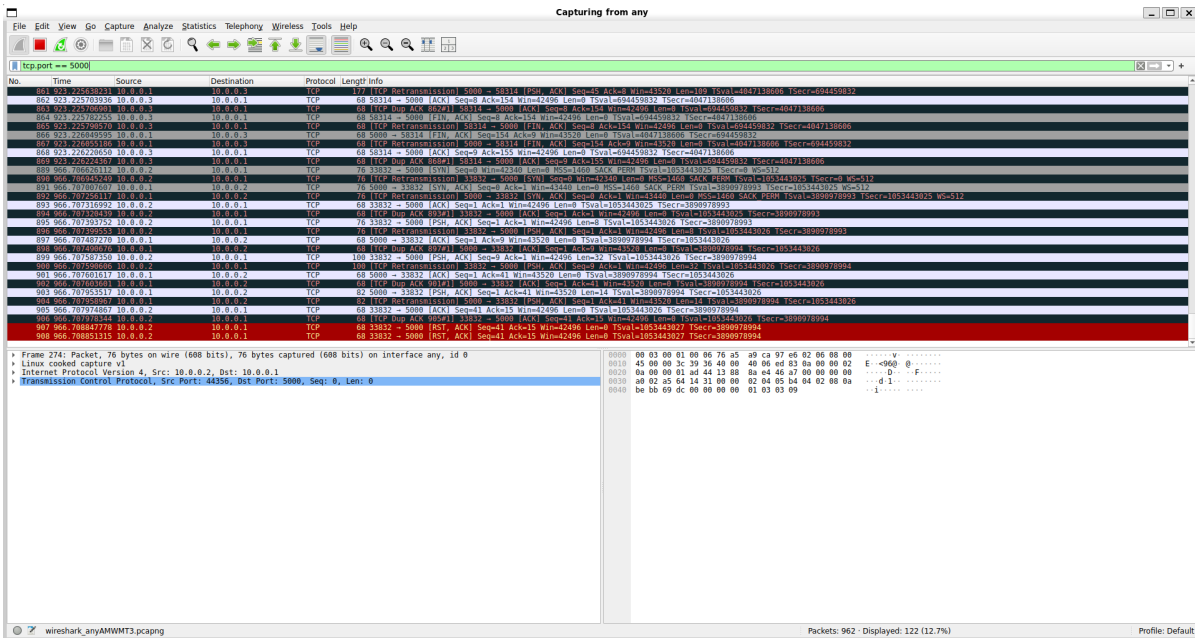


Figure 8.3: PSH-ACK frames across concurrent client connections carrying private message payloads.

Connection Close

TCP RST-ACK teardown frames were captured, confirming the connection was terminated with no orphaned client sockets left open.

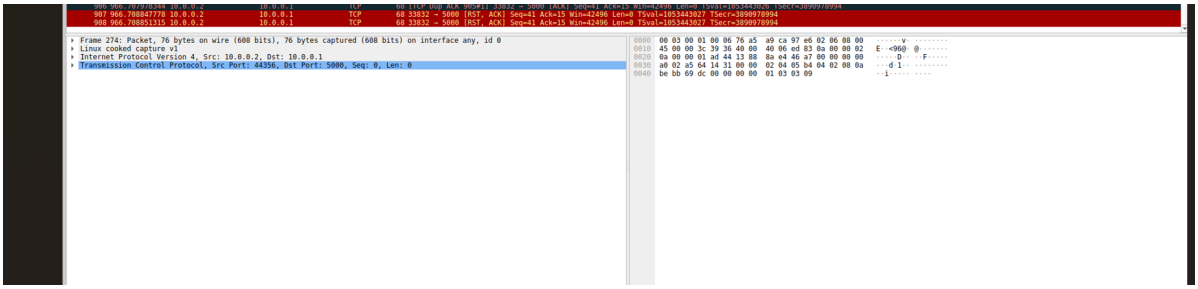


Figure 8.4: RST-ACK teardown frames confirming connection closure.

9. Reflection Answers

Q1. Why is server-side routing necessary for private messaging?

In a star topology, clients have no direct peer-to-peer TCP links between one another. The centralized server must look up the target user's active socket endpoint and forward the message selectively, since it is the only node with visibility into every connection.

Q2. How does maintaining user state improve the application?

Tracking active status, socket references, and login metadata for each client enables precise unicast routing, fast responses to the /list command, and clean broadcasting of disconnection events to the rest of the users.

Q3. How would the application scale to 100 users?

Per-client OS threads would need to be replaced with asynchronous I/O multiplexing (select/epoll, or Python's asyncio) or a bounded worker-thread pool, reducing memory consumption and context-switching overhead as the number of concurrent connections grows.

Q4. Which Wireshark packets verified private messaging?

A TCP PSH-ACK frame from the sender to the server, followed immediately by a single TCP PSH-ACK frame from the server to the targeted recipient's port only, confirming that no duplicate frames were sent to unrelated clients.

Q5. What improvements would you implement next?

End-to-end payload encryption using TLS/SSL sockets, separation of chat into multiple rooms/channels, user password authentication, and persistent storage in a database such as SQLite or PostgreSQL.

10. Conclusion

The enhanced multi-client TCP chat server was successfully implemented and verified within the Mininet network environment. Empirical testing confirmed correct operation of private message routing, detailed active-state tracking, history retrieval on reconnection, low message delivery latency, and stable server throughput across all tested concurrency levels.